

Measuring Validator Utility in Generate–Verify–Repair NetOps Pipelines: a Confirmatory Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (AC-2026-03) to evaluate whether a multidimensional validator metric suite predicts end-to-end agentic NetOps success better than a single verifier pass rate. Using a synthetic intent→configuration generator and a hosted generate–validate–repair (GVR) adapter under shared_initial_candidate semantics, we ran 200 paired tasks (one shared initial generation per task reused for baseline and guarded). The deterministic validator (deterministic_netops_validator_v5, v5.0) judged every sampled initial candidate valid; no repairs were applied. Baseline = 200/200, guarded = 200/200, paired difference = 0.00 (bootstrap 95% CI [0.00, 0.00], exact McNemar two-sided $p = 1.0$; bootstrap_seed = 1729, resamples = 10000). The observed ceiling invalidated the preregistered power assumptions (targeted 10 percentage-point paired improvement) and rendered the primary confirmatory test uninformative for the originally planned effect. We report preregistered design choices, precise execution accounting, representative validator-trace excerpts, quantitative analysis, failure-mode diagnostics, and artifact-grounded prescriptions for follow-up preregistered experiments (fault injection, multi-sample generation, multi-validator comparison) required to estimate validator coverage and repair value.

I. INTRODUCTION

Automating NetOps workflows with generative models requires reliable mechanisms to avoid harmful, silent, or wrong-but-plausible actions. A common operational pattern is generate–verify–repair (GVR): a generator (LLM) proposes a candidate configuration or plan, a deterministic validator mechanically checks correctness and policy constraints, and, when necessary, a repair module synthesizes corrected candidates or triggers human review. Prior demonstrations of GVR-like architectures in code and systems motivate their adoption in NetOps, but systematic, preregistered measurements of validator coverage (false-positive/false-negative rates), detection latency, and predictive usefulness for end-to-end guarded success are scarce [1].

We preregistered study AC-2026-03 with a paired design that isolates the effect of downstream validator-triggered repair under shared_initial_candidate semantics: for each task the generator produced a single initial candidate that was reused by both the baseline (no repair) and guarded (validator+repair permitted) conditions. Under these semantics any paired difference can only arise down-

stream from validator-triggered repair, not from independent generator sampling. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline and the preregistered confirmatory hypothesis (H1) expected that a multidimensional validator-metric suite would explain more variance in end-to-end success than a single verifier pass rate. Our main contribution is a fully preregistered, artifact-archived experiment and an exact, transparent report of a degenerate confirmatory outcome (ceiling) with concrete, artifact-grounded prescriptions for follow-up designs that can measure validator utility.

II. RELATED WORK

This section synthesizes prior verified work that motivates our measurement focus and clarifies limits relevant to validator-metric evaluation in NetOps.

Generate–verify–repair (GVR) architectures and deterministic validators. Several recent works articulate and prototype GVR-style architectures that interpose mechanistic checks between stochastic generation and acceptance. Cadenas [1] formalizes the generate–verify–repair pattern in the context of code-generation for regulated environments and argues for deterministic acceptance criteria; independently, recent arXiv/system reports describe self-healing orchestrators and verifier-guided recovery in tool-augmented LLM systems [4]. These records consistently demonstrate that verifier layers can detect mechanically checkable defects (compilation failures, structural constraint violations) and can provide structured repair feedback to generators. However, the cited demonstrations focus on defect classes that are mechanically testable and do not establish comprehensive empirical coverage for complex NetOps semantics; as a result, these works motivate architecture adoption while leaving open quantification of validator false-negative coverage on semantic or intent-alignment failures.

Benchmarks and operational failure modes. Controlled benchmark efforts in AIOps/NetOps document substantive, operationally meaningful failure modes that complicate end-to-end automation. OpsEval and Fault-Bench-style suites highlight sensitivity to noisy or unreliable user tickets, wrong premises, and wrong-but-plausible outputs that may trigger

incorrect actions [2],[3]. These benchmark records show that end-to-end agentic success depends on (a) the generator’s factual and procedural correctness, (b) the validator’s capacity to detect the relevant defect classes, and (c) the repair mechanism’s ability to produce acceptable candidates. Crucially, benchmark findings imply that verifier effectiveness is conditional on validator coverage relative to the failure modes present in the task bank; controlled improvements reported in prior work emerge when validation coverage aligns with the dominant mechanical defect classes in the evaluated tasks, not necessarily when semantic intent mismatches dominate.

Measured benefits and demonstrated limits of verifier-augmented strategies. System reports and experimental prototypes indicate that tool-augmented orchestration (verifier-guided repair, auditable decision traces, self-healing orchestrators) reduces silent failures and improves guarded success in those controlled contexts where validators detect actionable faults [4]. The archived literature and system reports support an important qualification: verifier-driven gains depend strongly on validator completeness for the defect classes that actually occur. When validators are narrowly focused on syntactic or structural checks, they will miss correct-by-structure but incorrect-by-intent outputs; when validators are broad they may increase false positives and operational triage costs. Our experiment and artifact record treat these prior demonstrations as architectural motivation and emphasize that a standardized validator-metric suite (coverage, FP/FN rates, detection latency, operational-impact proxy) is required to quantify these trade-offs in NetOps GVR pipelines [1],[4].

Design and measurement gaps in prior work. The verified literature converges on two practical gaps motivating this study. First, while GVR prototypes show the pattern’s usefulness for mechanically testable errors, there is no widely adopted, empirically estimable validator-metric suite tailored to NetOps GVR flows that reports FP/FN rates and predictive power for operational impact (see CG2 in our synthesis). Second, existing benchmarks and prototypes often use single-sample or narrow sampling regimes; without controlled generator variability or explicit fault-injection, empirical estimates of validator FN-rate and repair utility remain unstable. These gaps explain why prior reports can demonstrate verifier benefits in specific controlled settings but cannot by themselves establish general-purpose validator effectiveness across heterogeneous NetOps failure modes [2],[3],[5].

Practical implications drawn from prior records. The literature implies specific experimental controls needed to measure validator utility: deterministic task banks with explicit fault classes (to estimate FN/FPR under known perturbations), multi-sample generator draws or explicit injection of generator errors (to reintroduce variance), and multi-validator or multi-model comparisons (to bound external validity). Prior survey and benchmark records recommend quantifying verification cost as part of evaluation—reporting verification coverage and detection latency alongside success rates—rather than treating a single pass/fail rate as the sole measure of safety or reliability [5]. Our preregistered metric suite and the follow-up

prescriptions (fault-injection, multi-sample generation, multi-validator comparison) are designed specifically to address these literature-identified needs without presuming that validator layers alone universally deliver operational improvements.

III. METHODOLOGY

Preregistration and primary design. The study preregistration (AC-2026-03) specified a paired confirmatory design using the `hosted_netops_gvr_v1` adapter family with `generation_semantics = shared_initial_candidate` and `initial_generation_calls_per_task = 1`. Planned `task_count = 200` pairs (`planned_episode_count = 400`), `planned_maximum_model_calls = 400`, and `model_scope = gpt-5-mini` via the provider bundle recorded as `'openai_responses'`. The preregistered primary estimand was `paired_success_rate_difference_guarded_minus_baseline`; the primary test was an exact two-sided McNemar with paired bootstrap percentile confidence intervals (`bootstrap_resamples = 10000`, `bootstrap_seed = 1729`). Targeted detectable effect was an absolute 0.10 paired improvement with $\alpha = 0.05$ and ~80% power under conservative planning assumptions.

Generator, tasks, and constraints. Tasks were drawn deterministically from `deterministic_netops_task_generator_v5` (`task_family = intent_configuration_repair_v1`). Each task encodes: an `initial_state`, an `expected_final_state`, an explicit intent string, `allowed_fields/allowed_touched_fields`, `workflow_policies` (e.g., `minimum_active_in_groups`, `must_be_down_for_fields`), and a `required_command_sequence`. Task selection followed a preregistered deterministic index rule (`challenging_workflow_stress_v1`). Contamination/exclusion rules (`duplicate SHA-256` and `embedding-similarity > 0.95`) were preregistered; the execution/analysis artifacts record that no contamination exclusions occurred (`analysis/contamination_summary.csv` empty).

Model and sampling configuration. The declared model version in `execution/model_configuration.json` is `"gpt-5-mini"`; execution accounting records `model_calls_used = 200` (one shared initial generation per pair) and `maximum_model_calls = 400` as preregistered. Important artifact-grounded note: `execution/model_configuration.json` records `temperature = null` (`temperature = null/unset`). Null/unset is explicitly not evidence of deterministic provider-side sampling and does not imply `temperature = 0`; provider sampling variability remains possible and is recorded in provider-call audit fields. Provider-call audit in `deterministic_reconciliation` reports `hosted_model_call_event_count = 200` and `stage_counts.shared_initial_generation = 200`, consistent with the preregistered single shared generation per task.

Validator and scoring. The deterministic validator used was `deterministic_netops_validator_v5` (`validator_version = 5.0`). The validator implements mechanistic intent-constraint checks and emits per-episode fields: `normalized_configuration`, `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_count/violation_codes`, `valid` (boolean),

repair_applied (boolean), and repair artifacts when applicable. The preregistered binary scoring rule (full_intent_constraint_validation) uses validator.valid as the success label for the confirmatory estimand. The pipeline permitted at most one repair call when validator rejected a candidate; repair events would be recorded with repair_applied = true and repair_provider_trace fields.

Execution, exclusions, and missingness. The preregistered maximum_attempts_per_call = 1 (no manual retries). The execution manifest reports completed_episode_count = 400, failed_episode_count = 0, and complete_pair_count = 200. The preregistered missingness and contamination plans were followed: confirmatory analyses used complete pairs only; artifacts include analysis/missingness_summary.csv and analysis/contamination_summary.csv. All raw responses, per-episode validator traces, provider-call traces, and analysis artifacts are archived in the execution bundle and referenced in this paper.

IV. RESULTS

Execution accounting and primary outcomes. The adapter completed the run exactly as preregistered: planned_episode_count = 400, completed_episode_count = 400, failed_episode_count = 0, and model_calls_used = 200 (one shared initial generation per pair). Provider-call audit recorded hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, cache_hit_count = 0, cache_miss_count = 200, and stage_counts.shared_initial_generation = 200 (deterministic_reconciliation.json). Contamination and missingness artifacts confirm no exclusions or missing episodes (analysis/contamination_summary.csv empty; analysis/missingness_summary.csv reports complete_pairs = 200 and zeros for all missingness categories).

Primary confirmatory result and exact accounting. The deterministic validator judged every sampled shared initial candidate as valid; no repairs were applied in any episode. Analysis artifacts report baseline_success_count = 200/200 (success_rate = 1.0) and guarded_success_count = 200/200 (success_rate = 1.0). The paired contingency table (analysis/paired_contingency_table.csv) is n_11 = 200, n_10 = 0, n_01 = 0, n_00 = 0. Consequently the paired_success_rate_difference_guarded_minus_baseline = 0.00. The prespecified exact McNemar two-sided test yields p-value = 1.0. The paired bootstrap percentile 95% confidence interval (bootstrap_resamples = 10000, bootstrap_seed = 1729; analysis/analysis_log.jsonl) is [0.00, 0.00]. These numeric artifacts (analysis/condition_summary.csv, analysis/paired_contingency_table.csv, analysis/analysis_log.jsonl) are archived and match the reported values.

Representative per-episode validator diagnostics (artifact-grounded). To illustrate why no repairs were triggered, Table and scoring artifacts contain the validator trace fields for every episode; a compact example (execution/scoring/task-000004-guarded.json) shows the key mechanistic outputs that were recorded for each episode:

```
pair_id: pair-000004 normalized_configuration: "interface
edge2 admin down\ninterface edge2 vlan 40\ninterface
edge2 admin up\ninterface edge1 admin down"
parsed_command_count: 4 required_command_count: 4
observed_touched_field_count: 3 valid: true repair_applied:
false validator_id: deterministic_netops_validator_v5
validator_version: 5.0
```

Equivalent validator_trace.validation_after.valid = true and repair_applied = false hold for every archived episode; the repair_provider_trace fields are null for all episodes (execution/scoring/*.json entries list repair_provider_trace_path = null). Representative tasks in the artifact_examples (task-000004, task-000005, task-000006) likewise show parsed_command_count equal to required_command_count and valid = true, demonstrating that, for these sampled seeds, generator outputs satisfied the validator’s mechanistic constraints exactly.

Secondary and exploratory diagnostics (uninformative under ceiling). Because repair_applied = false for all episodes, secondary metrics that depend on rejection or repair events are degenerate in this execution: per-validator false-positive or false-negative rate estimates cannot be computed from observed rejections; detection-latency and repair-invocation distributions are empty by design. The archived raw_results.jsonl and per-episode validator traces nevertheless provide the full data required to compute these metrics under alternative scoring rules or after applying the follow-up prescriptions (fault injection or multi-sample generation) described elsewhere in this paper.

Artifact-grounded diagnostic interpretation. Under shared_initial_candidate semantics any paired difference could only arise from validator-triggered repair. The degenerate all-valid outcome observed here is explicable from artifact-grounded evidence in three linked facts: (1) the deterministic task generator and the recorded generation seeds produced initial candidates that satisfied the validator’s mechanistic intent-constraint checks (as shown by per-episode validation_after summaries); (2) the single-sample-per-task sampling regime removed generator variability as a source of disagreement by design (execution/model_configuration.json and deterministic_reconciliation.json record one shared generation per task); (3) mechanistic validators operate on structural and policy checks and can be blind to semantic intent misalignment (correct-by-structure yet incorrect-by-intent) absent a separate semantic-intent model or operator feedback. These jointly explain why no repairs occurred and why the confirmatory test could not detect the preregistered 10-percentage-point improvement target.

Reproducibility pointers and archived artifacts. All numeric claims above are reproducible from the archived artifacts: execution/raw_results.jsonl (input_results_sha256 = 23f4cd010f3a7419eb21ef9b8caf4dbe7730552eaa3f3eaa22d2068cf3b5d4e3), execution/model_configuration.json (sha256 = a40e96e2...), analysis/paired_contingency_table.csv (sha256 = 7bc1bd2a...), analysis/condition_summary.csv (sha256 = 1cb072b4...), analysis/analysis_log.jsonl (sha256 = dd2dbf18...), and representative per-episode scoring

files (execution/scoring/task-*.json). These files show the per-episode validator.valid flags, repair_applied markers, shared_initial_candidate content and SHA-256s, and provider-call audit entries used to produce the summary statistics above. Because the observed ceiling outcome is entirely recorded in these artifacts, reanalysis under modified scoring rules or preregistered follow-up arms (fault injection, multi-sample generation, multi-validator comparison) can be performed without re-executing the original shared-initial generation stage.

V. DISCUSSION

Design implications of shared_initial_candidate semantics. The preregistered shared_initial_candidate choice isolates downstream validator effects by ensuring both conditions evaluate the same initial candidate for each pair. This isolation is scientifically valuable because any observed paired difference must derive from validator-triggered repair. It also concentrates the risk of a single-sample ceiling: if the generator sample aligns with the validator for all tasks, the design yields zero observed variance in the primary estimand, as occurred here. We emphasize this property because it determines what the paired estimator can and cannot measure: it cannot detect differences attributable to generator sampling or to alternative prompt constraints when the sampling semantics were shared.

Operational interpretation and validator scope. The deterministic validator (deterministic_netops_validator_v5) reliably enforces mechanistic intent-constraint checks (parsed_command_count, required_command_count, violation_codes, valid flag). However, artifact-grounded evidence here shows that correctness-by-structure does not imply semantic intent alignment under operator expectations. That limitation is intrinsic to validators that lack an external intent model or operator feedback; detecting semantic mismatches requires either richer intent models, human-in-the-loop signals, or tailored semantic validators. The present execution does not imply validators are unnecessary—rather, it demonstrates that in a synthetic bank and single-sample regime the validator had nothing mechanical to correct.

Expanded diagnostics grounded in execution artifacts. The archived execution provides explicit numeric diagnostics that clarify why the confirmatory estimator was uninformative: model_calls_used = 200 (one shared generation per pair), complete_pair_count = 200, baseline_success_count = 200, guarded_success_count = 200, and repair_applied = false observed for every episode. The paired contingency is a point mass ($n_{11} = 200$), and the bootstrap procedure (bootstrap_seed = 1729, resamples = 10000) produced a degenerate sampling distribution with 95% percentile CI [0.00, 0.00]. These exact values are available in the analysis artifacts (condition summary and paired contingency outputs) and show that the confirmatory null is not a statistical fluke but a deterministic consequence of the joint generator-validator-sampling configuration used.

Practical lessons for preregistered validator evaluation. From a methodological perspective, two trade-offs are clear

and artifact-evident. First, shared_initial_candidate is a powerful control: it guarantees that any observed paired improvement is attributable to downstream validator/repair logic alone. Second, that control increases sensitivity to ceiling effects when generation produces validator-compliant candidates for the chosen seeds. For studies whose primary aim is to measure validator coverage (FP/FN) and repair utility, the preregistered design should therefore include one or both of the following preregistered modifications to avoid degeneracy: (a) a controlled fault-injection arm that deterministically introduces a known fraction f of invalid cases (syntactic, configuration-logic, or semantic-intent swaps) into the evaluation set so that the validator will face realistic defects; (b) a multi-sample generation arm with initial_generation_calls_per_task ≥ 3 independent draws per task to reintroduce generator variability and produce candidate diversity that can trigger validator rejections and repairs. Both prescriptions were articulated in the archived post-run prescriptions and are directly implementable within the same adapter semantics.

What validator metrics become estimable under those modifications. If faults are injected or multiple independent draws are allowed, the validator-metric suite preregistered in AC-2026-03 becomes estimable: (i) per-validator false-positive rate (FP-rate) as the fraction of validator.accept = true when the task’s ground-truth is invalid; (ii) per-validator false-negative rate (FN-rate) as validator.reject = false for candidates that do actually violate the required intent sequence; (iii) mean detection latency measured from model-response receipt to validator decision (records include timestamps in provider- and validator-traces); (iv) repair-invocation rate and post-repair success fraction when repair_applied = true is observed. These metrics map directly to the fields already emitted by the validator in the archived traces (valid, violation_count, repair_applied, parsed_command_count) and to provider-call audit counters; no additional opaque instrumentation is required to compute them once non-degenerate data are present.

Reproducibility and artifact-grounded protocol refinements. The execution artifacts document concrete accounting fields that support exact replication of the intended follow-ups: provider-call audit counts (hosted_model_call_event_count = 200), the model identifier (declared_model_version = gpt-5-mini), the recorded temperature = null/unset (explicitly not evidence of deterministic sampling), and the validator identifier/version (deterministic_netops_validator_v5, v5.0). A follow-up preregistration should (and can, per the archived infrastructure) explicitly set: (1) initial_generation_calls_per_task = k (e.g., $k = 3$) with independent generation seeds recorded for each draw; (2) a deterministic fault-injection schedule specifying fault classes and the exact fraction f of tasks affected; and (3) a plan to measure detection latency using the existing timestamps. These concrete protocol elements are already supported by the adapter and manifest fields and would remove ambiguity about sampling and allow FP/FN estimation without inventing new instrumentation.

Threats to validity revisited with artifact evidence. Inter-

nal validity: the `shared_initial_candidate` design intentionally removes generator sampling as a confound but introduces a ceiling risk; the exact accounting shows that risk materialized (complete agreement across conditions). Construct validity: `validator.valid` captures mechanistic constraint satisfaction but not semantic intent alignment—this remains the central construct gap and explains why correct-by-structure outputs can still be operationally unacceptable. External validity: the study executed one model (`gpt-5-mini`) and one deterministic validator against a synthetic task bank; therefore caution is required when extrapolating to multi-vendor production networks or to models and validators with different coverage. Conclusion validity: the archived bootstrap and contingency outputs (seed/resamples specified) demonstrate the degenerate inferential state (zero variance) and justify treating the confirmatory hypothesis as untestable under these run parameters rather than as a failed hypothesis test.

Operational implications and recommended reporting practices. For experimenters and operators evaluating GVR pipelines, we recommend preregistering both (a) the sampling semantics (`shared_initial_candidate` vs independent draws) and (b) explicit anti-ceiling design elements (fault injection or multi-draw sampling) whenever the evaluation goal is to estimate validator coverage or repair utility. Empirically reporting the exact counts we used (`model_calls_used`, `complete_pair_count`, baseline/guarded success counts, `repair_applied` counts, bootstrap/seed/resamples) increases interpretability because readers can immediately diagnose whether a noninformative null arose from lack of validator failures or from insufficient sample diversity.

Summary. The execution artifacts and numeric diagnostics make the outcome clear: under the chosen `shared_initial_candidate` semantics, the combination of deterministic seeds, single-sample generation, and a mechanistic validator that checks structural constraints produced a ceiling effect (200/200 valid). That outcome is scientifically informative about the joint behavior of generator+validator under these settings and usefully constrains what conclusions can and cannot be drawn. It also supplies a concrete, artifact-grounded roadmap—fault injection, multi-sample generation, and multi-validator comparison—that will enable future preregistered studies to estimate the validator metrics of interest and to quantify repair utility in operationally meaningful conditions.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics constrained inference: baseline and guarded reused the same initial candidate per pair; any paired difference could only arise from validator-triggered repair (not from independent generator sampling).
- Synthetic task bank (difficulty 'very_hard'/'extreme') is not equivalent to heterogeneous production networks (multi-vendor devices, real ticket noise); external validity is limited.
- Single-model experiment: only `gpt-5-mini` was executed; no multi-model generality claims are made.

- No repairs were applied because all initial candidates passed the deterministic validator; therefore the study could not estimate repair effectiveness or validator false-negative behavior in this execution.
- Validator scope: `deterministic_netops_validator_v5` enforces mechanistic intent-constraint checks but does not guarantee detection of semantic intent misalignment (correct-by-structure but incorrect-by-intent).
- Recorded `execution/model_configuration.json` `temperature = null/unset` does not imply deterministic sampling; provider-side sampling behavior may vary and is documented in execution manifests.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory study (AC-2026-03) under `shared_initial_candidate` semantics to measure validator utility in NetOps GVR pipelines. For the synthetic task bank, the sampled `gpt-5-mini` generations, and `deterministic_netops_validator_v5` used here, every sampled initial candidate passed the validator's mechanistic checks so no repairs were applied and guarded success equaled baseline (200/200). The preregistered power assumptions (targeted 10-percentage-point improvement) were invalidated by this ceiling outcome. The result is artifact-grounded and reproducible from the archived bundle. We publish the exact execution and analysis artifacts to support replication and provide concrete, preregistered experiment prescriptions (fault injection, multi-sample generation, multi-validator comparison) that are required to produce estimable validator FP/FN behavior and repair value in operationally meaningful conditions.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model and provider: `gpt-5-mini` via provider bundle '`openai_responses`'. Orchestration/adaptor: `hosted_netops_gvr_v1`. Deterministic validator: `deterministic_netops_validator_v5` (`validator_version = 5.0`). Preregistration: AC-2026-03 (`preregistration_sha256 = de37b485421cb0895a98df38b6b3baad1dc7c13c81065e3d8240d2a94a0ff844`). Master prompt immutable reference: `provenance/master_prompt.txt`, SHA-256 = `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba`. Autonomy boundary: a human Research Director selected the broad topic family and approved the master prompt before the autonomy lock; after the lock the AI system autonomously performed literature review, preregistration, experiment design, execution, analysis, internal peer review and manuscript drafting without manual editing of technical content. Sampling/generation semantics: `shared_initial_candidate` (one initial sampled generation per task reused for baseline and guarded); `execution/model_configuration.json` recorded `temperature = null/unset` (`null/unset` is not evidence of deterministic sampling and does not imply `temperature = 0`). Call budget and usage (preregistered): `maximum_model_calls = 400`; actual `model_calls_used = 200` (one shared initial generation per pair). All raw outputs, per-episode validator traces, execution

manifests, and analysis artifacts are archived in the supplied execution bundle (see execution/execution_manifest.json). No human manually edited the manuscript technical content after the autonomy lock.

REFERENCES

- [1] [1] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN Electronic Journal, 2026. doi:10.2139/ssrn.6754899.
- [2] [2] K.-H. Tseng, N. Bogahawatta, Y. Ginige, K. Patel, K. Dakic, S. Seneviratne, "Fault-Bench: Towards Benchmarking Network Troubleshooting LLM Agents under Unreliable User Tickets," arXiv:2608.27021, 2026.
- [3] [3] Y. Liu, C. Pei, L. Xu, B. Chen, M. Sun, Z. Zhang, Y. Sun, S. Zhang, K. Wang, H. Zhang, J. Li, G. Xie, X. Wen, X. Nie, M. Ma, P. Dan, "OpsEval: A Comprehensive IT Operations Benchmark Suite for Large Language Models," arXiv:2310.07637, 2023.
- [4] [4] R. S. Babu and A. Agrawal, "Self-Healing Agentic Orchestrators for Reliable Tool-Augmented Large Language Model Systems," arXiv:2606.01416, 2026.
- [5] [5] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, J. Zhang, X. Wang, J. Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities," IEEE Commun. Surveys & Tutorials, 2024. doi:10.1109/COMST.2024.3465447.